

DESARROLLO DE APLICACIONES MÓVILES

REFERENCIA DE WIDGETS Y DART PARA EL EXAMEN PRÁCTICO

Versión ajustada · consulta de sintaxis e intención

Qué hace esta guía

Recuerda sintaxis, propósito y uso mínimo de herramientas de Flutter y Dart. No decide por ti qué estado necesita el problema, qué reglas debes validar, cómo organizar una selección múltiple ni cómo calcular resultados del examen.

Qué NO encontrarás aquí

No hay patrones de selección múltiple, lógica de validación, conteos por categoría, sumas de valores, filtrados, búsquedas de máximos, colecciones de valores únicos ni código de la pantalla de resumen.

Mapa rápido de widgets

Necesidad	Herramienta
Organizar en horizontal	Row
Organizar en vertical	Column
Usar el espacio restante	Expanded
Separar del borde	Padding
Separar widgets	SizedBox
Agrupar información	Card
Construir muchas filas	ListView.builder
Zona completa tocable	InkWell
Acción con icono	IconButton
Valor sí/no visible	Checkbox [adicional]
Acción principal	FilledButton / ElevatedButton
Abrir otra pantalla	Navigator.push + MaterialPageRoute
Volver	Navigator.pop
Imagen local	Image.asset
Contenido alto con scroll	SingleChildScrollView
Dato mutable que afecta la UI	StatefulWidget + setState
Operación que tarda	Future + async/await

Leyenda

[Clase] fue trabajado directamente. [Adicional] puede usarse con ayuda de esta referencia, pero no es obligatorio.

1. Estructura y layout

Scaffold [Clase]

Úsalo cuando: quieres la estructura principal de una pantalla Material.

```
return Scaffold(
  appBar: AppBar(
    title: const Text('Ajustes'),
  ),
  body: const Center(
    child: Text('Contenido'),
  ),
);
```

Claves: Proporciona zonas como appBar y body; no contiene por sí solo la lógica del problema.

Row [Clase]

Úsalo cuando: necesitas colocar hijos uno al lado del otro.

```
Row(
  children: const [
    Icon(Icons.person),
    SizedBox(width: 8),
    Text('Ana'),
  ],
)
```

Claves: Su eje principal es horizontal.

Column [Clase]

Úsalo cuando: necesitas colocar hijos uno debajo del otro.

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: const [
    Text('Nombre'),
    Text('Descripción'),
  ],
)
```

Claves: crossAxisAlignment.start alinea los hijos al inicio del eje transversal.

Expanded [Clase]

Úsalo cuando: un hijo de Row o Column debe ocupar el espacio que queda disponible.

```
Row(
  children: const [
    Icon(Icons.article),
    SizedBox(width: 8),
    Expanded(
      child: Text(
        'Un texto que podría ser bastante largo',
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
      ),
    ),
  ],
)
```

Claves: Es frecuente usarlo alrededor de texto dentro de un Row.

Cuidado: No resuelve reglas de negocio; solo distribución de espacio.

Padding [Clase]

Úsalo cuando: quieres separar un hijo de los bordes.

```
const Padding(
  padding: EdgeInsets.all(16),
  child: Text('Contenido'),
)
```

SizeBox [Clase]

Úsalo cuando: necesitas una separación fija o tamaño controlado.

```
const SizeBox(height: 12)
```

Card [Clase]

Úsalo cuando: quieres agrupar visualmente una unidad de información.

```
Card(
  child: Padding(
    padding: const EdgeInsets.all(12),
    child: const Text('Información'),
  ),
)
```

Claves: La composición interna puede usar Row, Column, Expanded, etc.

SingleChildScrollView [Clase]

Úsalo cuando: una pantalla con un único bloque puede crecer más que la altura disponible.

```
SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: const [
      Text('Sección 1'),
      SizeBox(height: 20),
      Text('Sección 2'),
    ],
  ),
)
```

Cuidado: Para listas de muchos elementos, normalmente corresponde ListView.builder.

2. Listas visuales y texto

ListView.builder [Clase]

Úsalo cuando: tienes una colección y quieres construir un widget para cada posición.

```
final cities = ['Medellín', 'Bogotá', 'Cali'];

ListView.builder(
  itemCount: cities.length,
  itemBuilder: (context, index) {
    final city = cities[index];

    return ListTile(
      title: Text(city),
    );
  },
)
```

Claves: itemCount define cuántas posiciones se construyen; index identifica la posición actual.

Cuidado: El builder debe usar el dato de la posición actual, no un índice fijo.

Text + TextStyle [Clase]

Úsalo cuando: necesitas mostrar texto con jerarquía visual.

```
const Text(
  'Título',
  style: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
  ),
)
```

maxLines + TextOverflow.ellipsis [Clase]

Úsalo cuando: un texto largo debe ocupar un máximo de líneas.

```
const Text(
  'Descripción que puede crecer demasiado...',
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)
```

3. Interacción

InkWell [Clase]

Úsalo cuando: una zona completa debe responder al toque.

```
InkWell(
  onTap: () {
    print('Tocado');
  },
  child: const Padding(
    padding: EdgeInsets.all(16),
    child: Text('Abrir'),
  ),
)
```

Claves: onTap es un evento/callback. El widget no decide automáticamente qué dato debe cambiar.

IconButton [Clase]

Úsalo cuando: quieres una acción pequeña representada por un icono.

```
IconButton(
  onPressed: () {
    print('Acción');
  },
  icon: const Icon(Icons.info_outline),
)
```

Checkbox [Adicional]

Úsalo cuando: quieres representar visualmente un bool como marcado/no marcado.

```
bool notificationsEnabled = false;

Checkbox(
  value: notificationsEnabled,
  onChanged: (value) {
    setState(() {
      notificationsEnabled = value ?? false;
    });
  },
)
```

Claves: value indica cómo se ve. onChanged recibe el valor nuevo.

Cuidado: Este ejemplo solo muestra un bool individual. No describe cómo representar varias selecciones.

FilledButton [Clase]

Úsalo cuando: necesitas una acción principal.

```
FilledButton(
  onPressed: () {
    print('Guardar');
  },
  child: const Text('Guardar'),
)
```

Botón habilitado / deshabilitado [Adicional]

Úsalo cuando: una acción depende de una condición que ya tienes calculada.

```
final bool formComplete = true;

FilledButton(
  onPressed: formComplete
    ? () {
        print('Enviar');
      }
    : null,
  child: const Text('Enviar'),
)
```

Claves: En botones Material, onPressed: null lo deja deshabilitado.

Cuidado: La guía no indica cómo calcular la condición de tu examen.

4. Estado y comunicación entre widgets

StatefulWidget + setState [Clase]

Úsalo cuando: un dato cambia durante la ejecución y ese cambio afecta la interfaz.

```
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  void increment() {
    setState(() {
      count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Text('Contador: $count');
  }
}
```

Claves: count es el estado. setState notifica que cambió información relevante.

Cuidado: setState no es el estado y no determina qué datos necesita tu problema.

VoidCallback [Clase]

Úsalo cuando: un hijo debe informar que ocurrió una acción sin conocer toda la lógica del padre.

```
class ActionTile extends StatelessWidget {
  final String label;
  final VoidCallback onTap;

  const ActionTile({
    super.key,
    required this.label,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(label),
      onTap: onTap,
    );
  }
}
```

Claves: VoidCallback equivale a void Function(): sin parámetros y sin valor de retorno.

Usar el callback desde el padre [Clase]

Úsalo cuando: el padre debe decidir qué ocurre cuando el hijo reporta el evento.

```
ActionTile(
  label: 'Ver información',
  onTap: () {
    print('El padre decide la acción');
  },
)
```

Claves: Idea general: datos bajan por parámetros; eventos suben mediante callbacks.

5. Navegación y paso de datos

Navigator.push + MaterialPageRoute [Clase]

Úsalo cuando: quieres abrir otra pantalla.

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => const SettingsPage(),
  ),
);
```

Claves: push agrega una ruta sobre la actual.

Pasar un objeto por constructor [Clase]

Úsalo cuando: el destino necesita un objeto que ya conoce la pantalla actual.

```
class PersonDetailPage extends StatelessWidget {
  final Person person;

  const PersonDetailPage({
    super.key,
    required this.person,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(person.name),
      ),
      body: Text(person.email),
    );
  }
}
```

Claves: La pantalla declara explícitamente qué dato necesita.

Navigator.pop [Clase]

Úsalo cuando: quieres cerrar la pantalla actual y regresar.

```
Navigator.pop(context);
```

6. Imágenes y assets

Image.asset [Clase]

Úsalo cuando: quieres mostrar una imagen local.

```
Image.asset(
  'assets/images/avatar.jpg',
  width: 80,
  height: 80,
  fit: BoxFit.cover,
)
```

Claves: La ruta debe coincidir con el archivo y la carpeta debe estar registrada.

pubspec.yaml [Clase]

Úsalo cuando: quieres registrar una carpeta de assets.

```
flutter:
  uses-material-design: true
  assets:
    - assets/images/
```

Cuidado: La indentación de YAML importa.

ClipRRect [Clase]

Úsalo cuando: quieres recortar el hijo con bordes redondeados.

```
ClipRRect(
  borderRadius: BorderRadius.circular(12),
  child: Image.asset(
    'assets/images/avatar.jpg',
    width: 80,
    height: 80,
    fit: BoxFit.cover,
  ),
)
```

BoxFit.cover [Clase]

Úsalo cuando: quieres que la imagen llene el espacio sin deformarse.

```
Image.asset(
  'assets/images/photo.jpg',
  width: 120,
  height: 80,
  fit: BoxFit.cover,
)
```

Claves: Puede recortar parte de la imagen para conservar proporción.

7. Asincronía y estados de proceso

Future + async / await [Clase]

Úsalo cuando: una operación tarda y su resultado llegará después.

```
Future<String> loadMessage() async {
  await Future.delayed(
    const Duration(seconds: 1),
  );

  return 'Datos listos';
}
```

Claves: await espera el resultado antes de continuar esa secuencia.

try / catch [Clase]

Úsalo cuando: la operación puede fallar y necesitas manejar el error.

```

try {
  final result = await loadMessage();

  setState(() {
    message = result;
    isLoading = false;
  });
} catch (e) {
  setState(() {
    errorMessage = 'No fue posible cargar';
    isLoading = false;
  });
}

```

Claves: Éxito y error deben dejar el estado coherente.

UI según estado [Clase]

Úsalo cuando: la pantalla puede representar diferentes situaciones.

```

Widget buildBody() {
  if (isLoading) {
    return const Center(
      child: CircularProgressIndicator(),
    );
  }

  if (errorMessage.isNotEmpty) {
    return Text(errorMessage);
  }

  return Text(message);
}

```

Claves: La UI lee el estado actual; no debería iniciar operaciones con efectos desde build.

8. Dart básico de consulta

Límite de esta sección

Solo recuerda sintaxis básica. No incluye métodos que resuelvan filtrados, búsquedas, sumas, conteos por categoría o valores únicos.

List<T> [Clase]

Úsalo cuando: necesitas almacenar varios valores del mismo tipo.

```

final List<String> names = [
  'Ana',
  'Luis',
  'Marta',
];

print(names.length);
print(names[0]);

```

Claves: length indica cantidad de elementos. [index] accede a una posición.

if / else [Clase]

Úsalo cuando: necesitas ejecutar caminos diferentes según una condición.

```
if (age >= 18) {
    print('Mayor de edad');
} else {
    print('Menor de edad');
}
```

for [Clase]

Úsalo cuando: necesitas recorrer valores uno a uno.

```
final names = ['Ana', 'Luis', 'Marta'];

for (final name in names) {
    print(name);
}
```

Claves: La lógica que ejecutes dentro del ciclo depende del problema.

bool [Clase]

Úsalo cuando: necesitas representar una condición true/false.

```
bool active = true;
```

final [Clase]

Úsalo cuando: una referencia se asigna una vez.

```
final String name = 'Ana';
```

required [Clase]

Úsalo cuando: un parámetro nombrado debe proporcionarse al construir el objeto.

```
const UserCard({
    super.key,
    required this.name,
});
```

Null safety: ? y ?? [Clase]

Úsalo cuando: un valor puede ser null y necesitas un valor alternativo.

```
int? score;

final visibleScore = score ?? 0;
```

Operador ternario [Clase]

Úsalo cuando: quieres elegir entre dos valores usando una condición sencilla.

```
final bool online = true;

final text = online
    ? 'En línea'
    : 'Desconectado';
```

Cuidado: Para reglas con varias condiciones, un if o una función suele ser más legible.

9. Diagnóstico rápido

Síntoma	Qué revisar
El dato cambia pero la UI no	¿El cambio relevante está dentro de setState? ¿build lee el dato actualizado?
Todas las filas muestran lo mismo	¿El builder usa el elemento correspondiente al index actual?
Todas las filas abren el mismo detalle	¿Se pasa el objeto actual o uno fijo?
Overflow horizontal	¿Hay texto largo dentro de Row? ¿Expanded o maxLines ayudan?
La imagen no aparece	¿Archivo, ruta, mayúsculas y pubspec coinciden?
Botón deshabilitado incorrectamente	¿La condición que le pasas representa el estado actual?
Loading permanente	¿isLoading vuelve a false tanto en éxito como en error?
La función se ejecuta muchas veces	¿Se está llamando desde build?

10. Antes de terminar

Recuerda

La documentación responde “¿cómo se usa esta herramienta?”. El examen responde “¿qué herramienta necesito, qué estado debo diseñar y cómo hago que mis reglas funcionen juntas?”.

- ☐ La pantalla representa datos reales y no valores fijos.
- ☐ Cada fila usa el elemento correspondiente.
- ☐ Los eventos llaman la lógica correcta.
- ☐ La navegación lleva el objeto correcto.
- ☐ El layout no presenta overflows.
- ☐ La UI cambia cuando cambia el estado.
- ☐ Probé un caso normal, uno inválido y un caso límite.